Skill Swap App

MOBILE APPLICATION DEVELOPMENT

Assignment 1: Plan, Design and Prototype

Course	Mobile Application Development
Assignment	Assignment 1: Plan, Design and Prototype (25 marks)
University	Pak Austria Fachhochschule Institute of Applied Sciences and Technology
Group	Group 14
App name	Skill Swap App
Date	1 November 2026

Group members

#	Name	Student ID
1	Muhammad Fuzail	B24S0540CS030
2	Rabia Rafaqat	B24S0230CS002
3	Javeria Imran	B24S0524CS003

GitHub repository: github.com/MuhammadFuzail288/Skill-Swap-App

1 Project Idea and Problem Statement

1.1 Project statement

Many university students have skills they could share, such as design, programming or languages, but cannot afford paid courses, while other students want to learn exactly those skills. Skill Swap App lets a student keep one organized, searchable record of the skills they can teach, the skills they want to learn and the peers they plan to swap with. Each entry can carry an optional session date and time, and the app sends a local reminder so planned learning sessions are not forgotten. Compared with scattered chat messages or paper notes, the app keeps everything structured, searchable and available offline on the student's own phone. Compared with paid course platforms, it focuses on free, peer-to-peer exchange of knowledge between students.

1.2 Target user

Primary: university students who want to exchange skills with peers instead of paying for courses.
Also useful for: fresh graduates, freelancers and self-learners.

1.3 Value proposition

For university students who want to learn new skills without paying for courses, Skill Swap App is a mobile app that lets them log skills they can teach and want to learn, plan swap sessions and get reminders, all offline on their own phone.

1.4 Example

Ali teaches Photoshop and wants Web Development; Sara teaches Web Development and wants Photoshop. Each logs the swap with a session time and gets a reminder.

1.5 Competitor review

Existing app	Strength	Weakness	How Skill Swap App differs
Skillshare	Large learning platform with professional classes	Paid subscription	Free and student-focused; peer-to-peer exchange instead of paid teaching; data stored on the device and usable offline; simple, single-purpose interface
Udemy	Large collection of courses	Courses must be purchased	

1.6 Scope note

Following the project rules, all data is stored **locally on the device** and there is **no login**. Each phone therefore holds its own skill board. The app does **not** match users across different devices. Cross-device matching and cloud sync are listed only as *Could* features (section 3). The single device integration is **local notifications**.

1.7 Rule compliance

Project rule	**How Skill Swap App meets it**
CRUD list of items	Skill posts: create, read, update, delete
Local data storage	Hive database on the device
Exactly one integration	Local notifications (session reminders)
5 to 8 screens; no login by default	6 screens; authentication listed as Could only
One platform	Android (Flutter, demonstrated on Android)

2. Requirements

2.1 Functional requirements (12)

ID	**Description**	**Priority**	**Built in**
FR-01	The user can add a skill post with name, skill offered, skill wanted, category, description and an optional session date and time.	**Must**	A3
FR-02	The user can view a list of all saved skill posts.	**Must**	A3
FR-03	The user can open a post to view its full details.	**Must**	A3
FR-04	The user can edit an existing skill post.	**Must**	A3
FR-05	The user can delete a skill post after confirming the action.	**Must**	A3
FR-06	The user can search skill posts by skill name.	**Must**	A3
FR-07	The user can filter skill posts by category (Design, Programming, Marketing, Language Learning).	**Must**	A3

FR-08	The user can sort skill posts by newest, oldest or alphabetical order.	**Should**	A4
FR-09	The system shall validate that name, skill offered and skill wanted are filled in before a post is saved.	**Must**	A3
FR-10	The system shall store all skill posts locally on the device using Hive.	**Must**	A3
FR-11	The system shall send a local notification reminder at the session date and time set on a post.	**Must**	A3
FR-12	The system shall show a clear empty-state message when there are no posts or no search results.	**Should**	A4

2.2 Non-functional requirements (4)

ID	Description (measurable)	Priority	Built in
NFR-01	The home screen loads in under 3 seconds on the test device or emulator.	**Must**	A4
NFR-02	All saved data remains available offline and after the app is restarted.	**Must**	A3
NFR-03	All touch targets are at least 48 x 48 dp.	**Must**	A2
NFR-04	Text and background colours meet a contrast ratio of at least 4.5:1.	**Must**	A2

2.3 Could features (not counted above, built only if all Must and Should items are finished)

ID	Description	Priority	Built in
C-01	Cloud backend with sync and cross-device matching of users.	**Could**	Not planned
C-02	User accounts and authentication.	**Could**	Not planned

3. Technology Stack and Justification

3.1 Chosen stack

Component	Choice	Role in the project

Framework	**Flutter**	Builds all screens, navigation and forms from one codebase
Language	**Dart**	Single language for UI, data access and logic
Data storage	**Hive** (local NoSQL database)	Stores skill posts on the device; no SQL required
Notifications	Flutter local-notifications package (final package confirmed in A3)	Schedules the session reminder (the single integration)
IDE	**Android Studio** with Flutter and Dart plugins	Coding, debugging and running the app
Test target	Android Emulator and/or physical Android phone	Running and demonstrating the app on Android
Version control	**Git and GitHub**	Shared repository, branches and commit history

3.2 Why this stack

- **Fits the app's needs.** The app is built from lists and forms. Flutter's built-in widgets (lists, forms, bottom navigation, Material components) cover all six screens, and Hive stores post objects directly
- on the device. **One language, one codebase.** Using Dart for everything keeps the project easy to understand for three members and lets us split work by screen or feature. Hot reload speeds up UI
- iteration in Assignment 2.

Realistic for 8 to 10 weeks. Flutter has extensive official documentation, and Hive needs very little setup, which suits the limited scope of CRUD, search and one reminder feature.

3.3 Comparison with alternatives

Criteria	**Flutter + Dart + Hive chosen**	**React Native + JavaScript + AsyncStorage**	**Kotlin + Jetpack Compose + Room**
Learning curve	Moderate: one new language (Dart) and one consistent widget system	Moderate: easier with prior JavaScript and React knowledge	Steeper: Kotlin plus Androidspecific concepts
Performance	Good: compiled to native code; smooth for list and form apps	Good for CRUD apps	Best: fully native
Community support	Large; official docs and pub.dev packages	Large; npm ecosystem	Large; official Android documentation
Platform coverage	Android and iOS from one codebase	Android and iOS	Android only
Local storage	Hive: simple object boxes	AsyncStorage: key-value; extra work for structured lists	Room: powerful SQLite layer, more boilerplate

Fit for this app	**High**: all screens, CRUD and local reminders with a small set of packages	Medium to high	Medium: strong, but Androidonly and heavier setup

4. Technology Stack

4.1 Classification: cross-platform

Skill Swap App is a **cross-platform** app. One Dart codebase is compiled into a native Android app, and the same code could target iOS later. It is not a hybrid (WebView) app, because Flutter draws its own interface rather than showing web pages inside a shell. For this project it means we write and maintain one app and demonstrate it on **Android**, as the project rules allow.

4.2 Data model

Field	**Type**	**Description**
id	Integer	Unique key of the post
name	Text	Name of the person in the exchange
offeredSkill	Text	Skill the person can teach
wantedSkill	Text	Skill the person wants to learn
description	Text	Extra details
category	Text	Design, Programming, Marketing or Language Learning
createdDate	DateTime	When the post was created
sessionDateTime	DateTime (optional)	Planned session time; triggers the local reminder

4.3 How the stack covers each requirement area

Requirement area	**Covered by**
Screens, forms and navigation (all screens)	Flutter widgets: lists, forms, bottom navigation bar and stack navigation

Create, read, update, delete (FR-01 to FR-05)	Hive box skill_posts with the SkillPost model
Search, filter and sort (FR-06 to FR-08)	Dart list operations on the posts loaded from Hive
Local storage and offline use (FR-10, NFR-02)	Hive stores data on the device; no network is needed
Session reminder (FR-11)	Local-notifications package for Flutter, using sessionDateTime

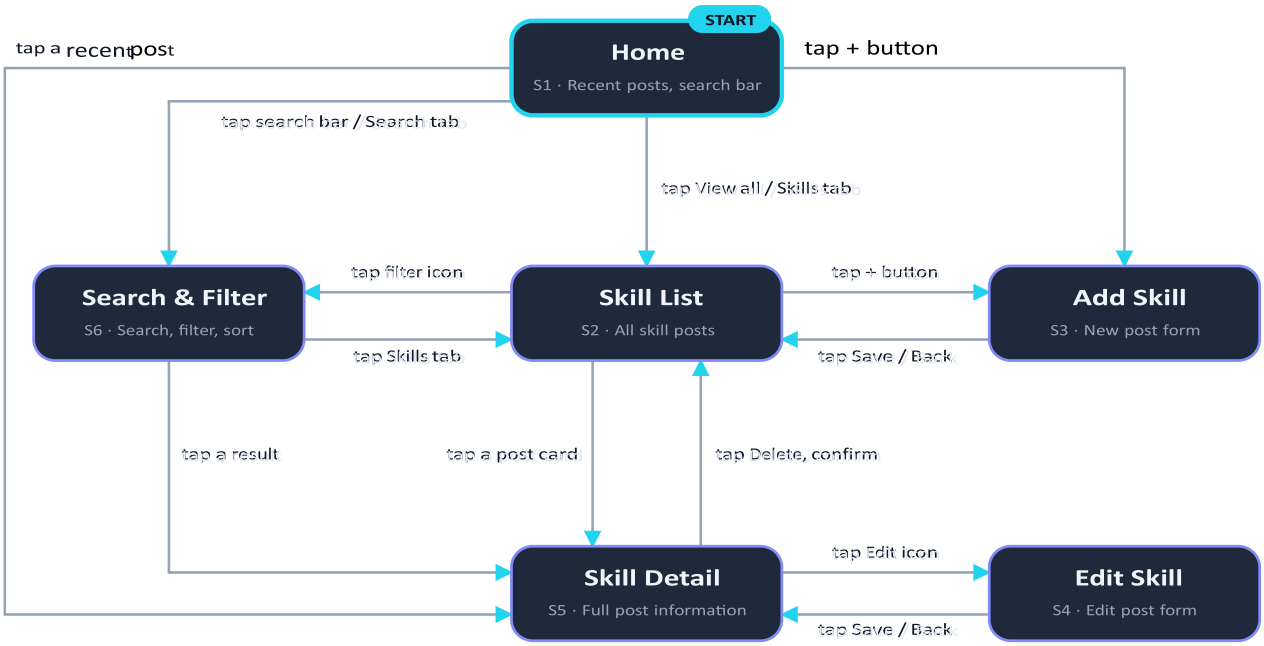
4.4 Planned app structure

Planned folder layout for the Flutter project (to be created in Assignment 2).

Path	**Purpose**
lib/main.dart	App entry point and navigation setup
lib/theme/app_theme.dart	Colours, Poppins typography and component styles from the style guide
lib/models/skill_post.dart	SkillPost model (fields in 4.5)
lib/data/skill_repository.dart	Hive access: add, get, update, delete, query
lib/screens/	Home, Skill List, Add Skill, Edit Skill, Skill Detail, Search & Filter
lib/widgets/	Reusable skill card, form fields, buttons, empty-state view

5.Design

5.1 Screen flow diagram



Navigation

Start screen: Home (S1).
Pattern a bottom tab bar (Home, Skills, Search) is visible on the three main screens, and stack navigation opens Add, Detail and Edit on top. The tab bar links Home (S1), Skill List (S2) and Search & Filter (S6) at all times.

Screens (6)

S1 Home · S2 Skill List · S3 Add Skill · S4 Edit Skill · S5 Skill Detail · S6 Search & Filter. The + button is available on Home and Skill List; Back returns to the previous screen from every screen.

User-centred design principles applied

Know the user: students who want a quick, simple record, so every form has only the fields the exchange needs.

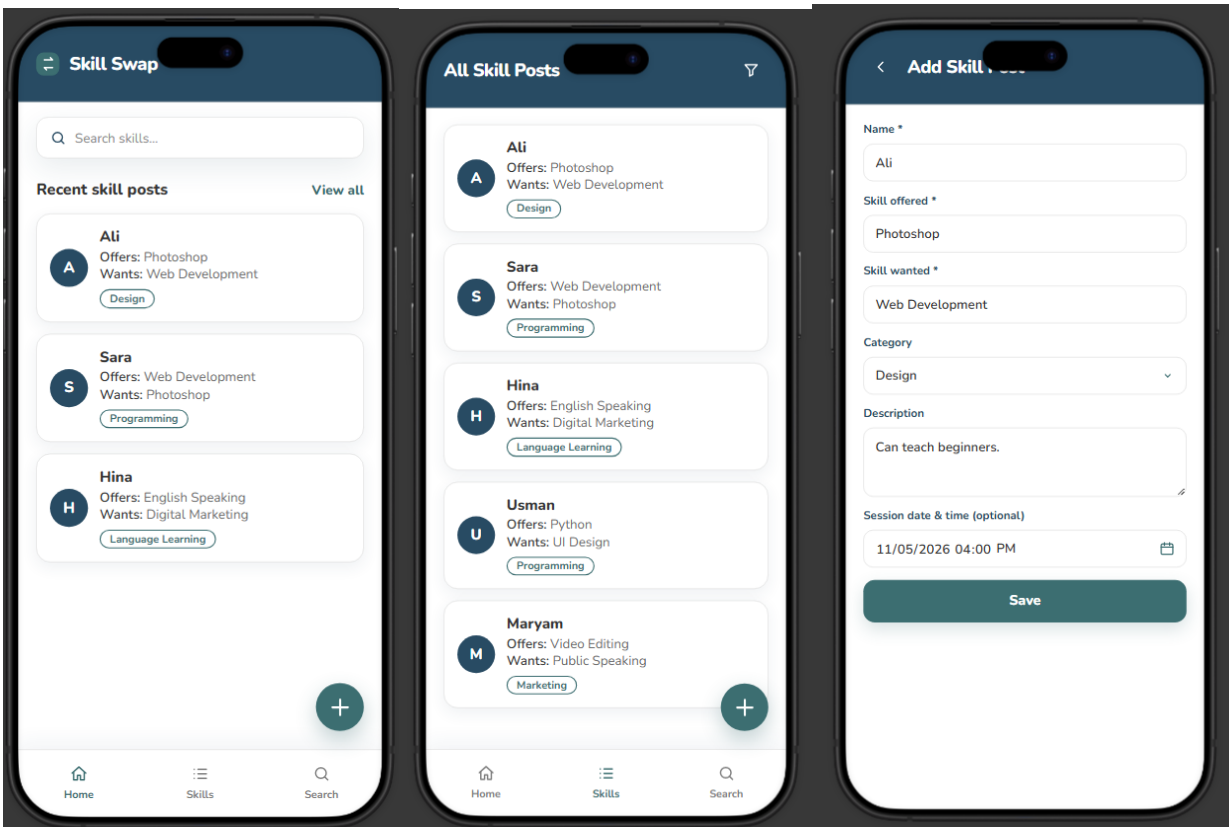
Simple navigation: three tabs, one + button, and no more than two taps to reach any screen.

Consistency: the same post card, form fields, buttons and colours on every screen.

Thumb reach and small screens: the tab bar and + button sit at the bottom of the screen, and all touch targets are at least 48 x 48 dp (NFR-03).

5.2 Wireframes (1 of 2)

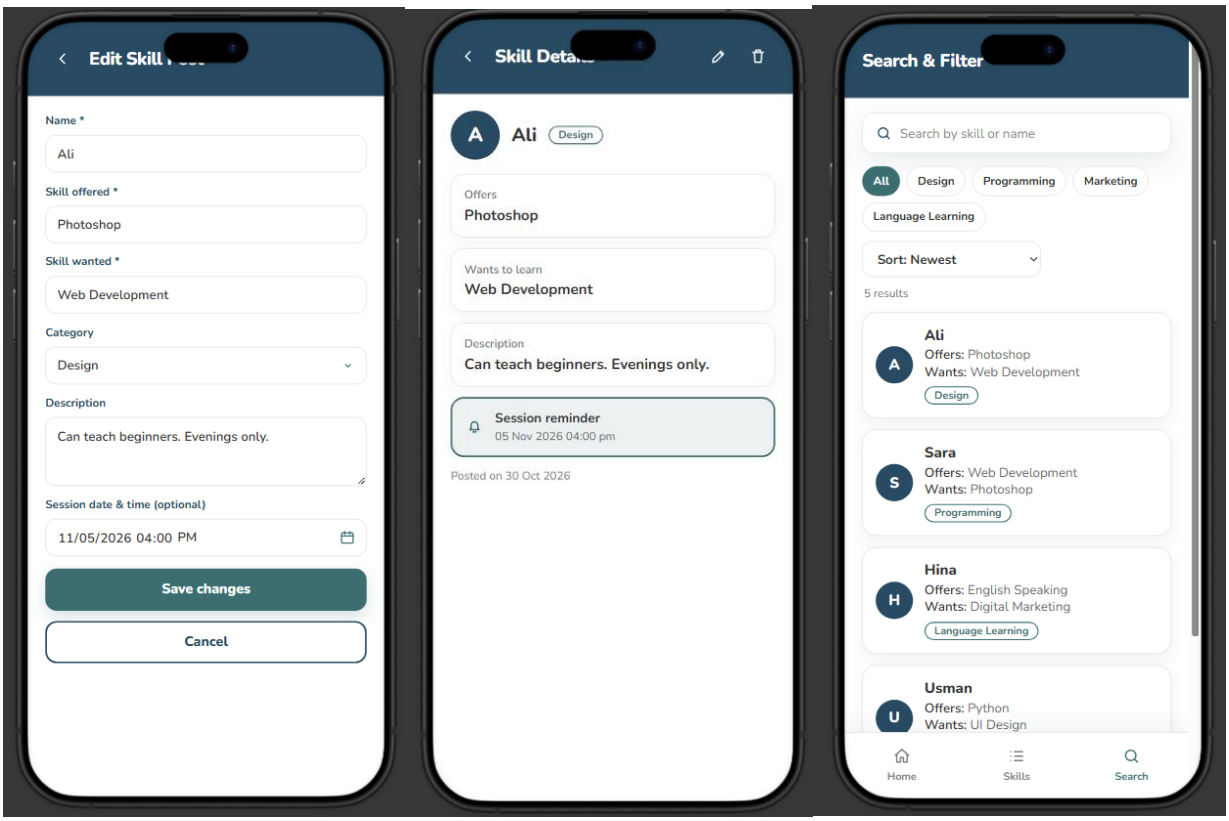
Mobile proportions (about 1:2). Numbered markers identify the main interactive elements.



1. **HOME SCREEN**
   - Search bar: opens search and filter
   - View all: apen skill list
   - Recent post card: opens skill details
   - + button: opens add skill
2. **ALL SKILL POST SCREEN**
   - Filter icon: opens search and filter
   - Post card: opens skill detail
   - + button: opens add skills
3. **ADD SKILL POST SCREEN**
   - Back: return without saving
   - Category: drop-down list of four categories
   - Session date and time: optional; schedules the reminder
   - Save: validate required field(*), saves, returns to the list

5.2 Wireframes (2 of 2)

Edit uses the same form as Add, pre-filled with the saved post.



4. **EDIT SKILL POST SCREEN**
    - Back: return to skill detail
    - Category: drop down list
    - Save changes: validates, update the post, return to skill detail
    - Cancel: discard changes
5. **SKILL DETAIL SCREEN**
    - Edit icon: open Edit skills
    - Delete icon: asks for confirmation then remove the post
    - Session reminder: showed the planned date and time

6. **SEARCH AND FILTER SCREEN**
    - Search field: searches by skill name
    - Category chips: filter by category
    - Sort drop-down: Newest, Oldest, Alphabetically
    - Result car: opens skill detail

5.3 Style guide Color

#3C6E71 Primary Buttons, active tab, + button, avatars	**#353535** Accent Chips, links, highlights, markers	**#FFFFFF** Background All screens

Supporting neutrals: Skill cards: white, with a light-gray border and charcoal text. Headings, names and body text**#353535 — Charcoal**, App header, secondary actions and emphasis**#284B63 — Deep blue**, Borders, dividers and subtle section backgrounds **#D9D9D9 — Light gray**.

Contrast check (WCAG, minimum 4.5:1 for text)

Text color	Background	Ratio	Result
Charcoal `#353535`	White `#FFFFFF`	12.27 : 1	Pass
Charcoal `#353535`	Light gray `#D9D9D9`	8.69 : 1	Pass
White `#FFFFFF`	Muted teal `#3C6E71`	5.74 : 1	Pass
White `#FFFFFF`	Deep blue `#284B63`	9.22 : 1	Pass
Muted teal `#3C6E71`	White `#FFFFFF`	5.74 : 1	Pass
Deep blue `#284B63`	White `#FFFFFF`	9.22 : 1	Pass
Deep blue `#284B63`	Light gray `#D9D9D9`	6.53 : 1	Pass

Typography: Poppins

Style	Size	Weight	Used for
Heading	22 sp	SemiBold	Screen titles
Subheading	16 sp	SemiBold	Section titles, names on cards
Body	14 sp	Regular	Skill text, form fields, descriptions
Caption	12 sp	Regular	Dates, hints, chip labels

6. Project Setup

6.1 Repository

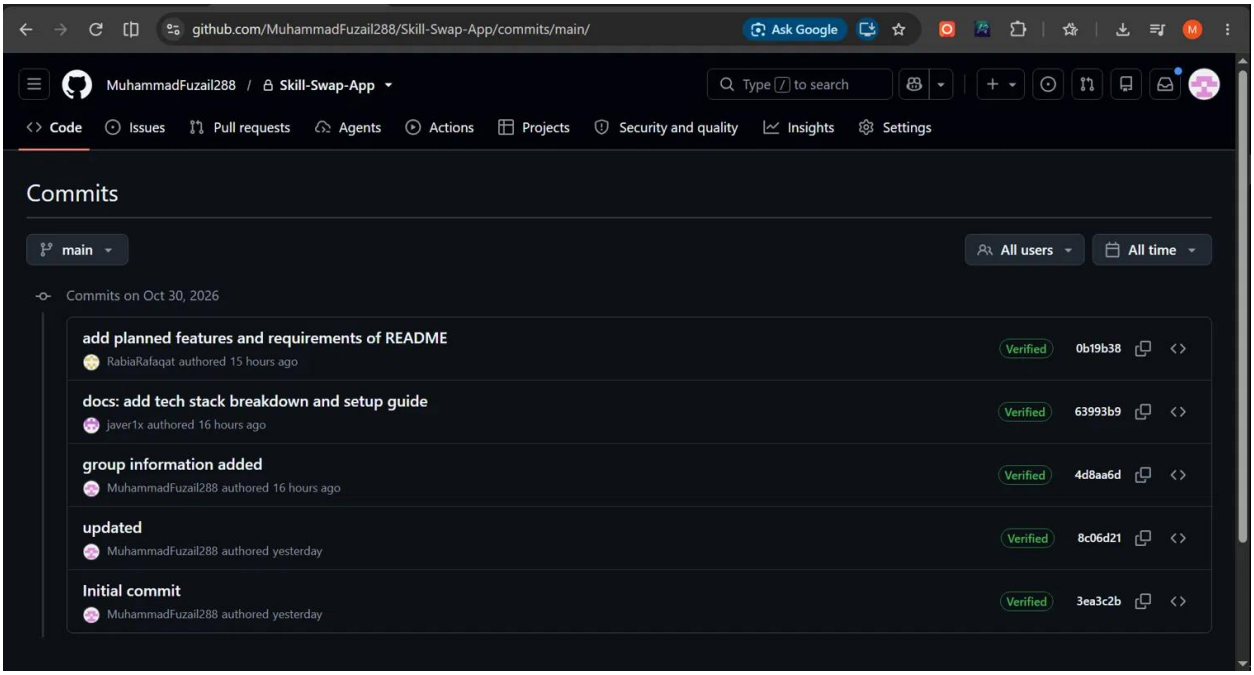
Repository link	https://github.com/MuhammadFuzail288/Skill-Swap-App
Clone URL	`https://github.com/MuhammadFuzail288/Skill-Swap-App.git`

Structure	`docs/` holds the report, wireframes and diagrams; the Flutter app source sits in the standard project folders.

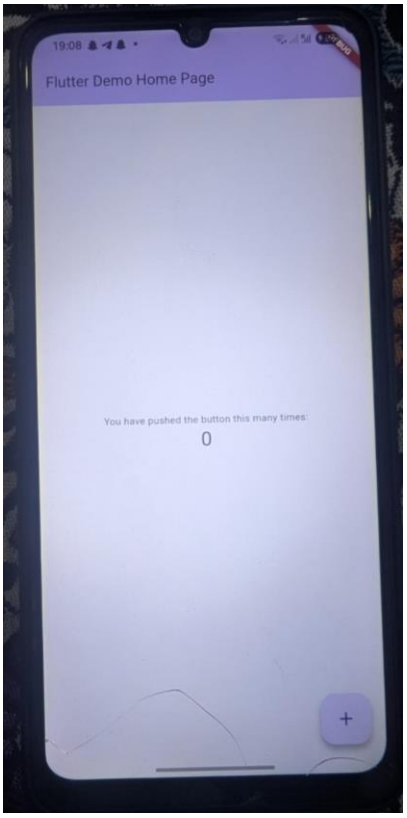
6.2 README summary

The README.md in the repository contains: app name, group name, members with names and IDs, project description, planned features (Must / Should / Could), the chosen tech stack, and setup instructions (Flutter and Android Studio prerequisites, clone, flutter pub get , flutter run).

6.3 Commit History



6.4 Hello world running app



6.5 Setup steps followed

- Installed the Flutter SDK and Android Studio with the Flutter and Dart plugins.
- Created the Flutter project and ran the default app on an Android emulator or phone.
- Made the initial commit and pushed it to the GitHub repository.
- Each member made at least one commit to the repository before the submission date.

7. Work Plan for Assignments 2 to 4

7.1 Task list per assignment

A2: User Interface and Navigation (all six screens with dummy data; NFR-03 and NFR-04)

Member	Tasks	Screens
Muhammad Fuzail	Project folder structure; bottom tab bar and stack navigation; app theme from the style guide (colours, Poppins, icons)	Home
Rabia Rafaqat	Reusable skill post card; list and detail layouts; empty-state layout	Skill List, Skill Detail

Javeria Imran	Reusable form fields and buttons; Add and Edit form layouts; search field, category chips and sort control	Add, Edit, Search & Filter

A3: Data and Functionality (FR-01 to FR-07, FR-09, FR-10, FR-11, NFR-02)

Member	Tasks	Requirements
Muhammad Fuzail	Hive setup and SkillPost model; data access layer; connect list and detail to real data	FR-02, FR-03, FR-10, NFR-02
Rabia Rafaqat	Create, edit and delete flows with confirmation; form validation	FR-01, FR-04, FR-05, FR-09
Javeria Imran	Search by skill name; category filter; local notification scheduling for session reminders	FR-06, FR-07, FR-11

A4: Testing, Polish and Delivery (FR-08, FR-12, NFR-01)

Member	Tasks	Requirements
Muhammad Fuzail	Data layer tests; performance check of home screen load time; release APK build; final documentation	NFR-01
Rabia Rafaqat	Sort feature; empty states and error handling; widget tests	FR-08, FR-12
Javeria Imran	Device testing; test report; demo video; presentation slides	Verification of all requirements

All three members work together on the final presentation and on bug fixing.

7.2 Timeline

Phase	Relative weeks	Main milestones
A2: UI and Navigation	Weeks 1 to 3	Week 1: theme, navigation, shared components. Week 2: all six screens. Week 3: review, fix layout and style issues.
A3: Data and Functionality	Weeks 4 to 6	Week 4: Hive and SkillPost model, notification test on a device. Week 5: CRUD, validation, search and filter. Week 6: integration and bug fixing.
A4: Testing, Polish and Delivery	Weeks 7 to 9	Week 7: sort, empty states, error handling. Week 8: tests and device testing. Week 9: release APK, demo video, final documentation, presentation.

7.3 Risk list

#	Risk	How we will reduce it
1	Local notifications do not fire reliably on some Android versions (for example because of the notification permission).	Build and test a minimal notification example on a physical phone in the first week of A3; request the permission when the user first sets a session time.
2	Scope grows (cloud sync, matching, login).	Scope is fixed to the requirements table. Could features start only after all Must and Should items are finished.
3	Merge conflicts or uneven contribution in the repository.	Feature branches, small commits, pull before push, and review of each pull request by another member (see working agreement).
4	Flutter or emulator setup problems on a member's computer.	Follow the README setup steps and run `flutter doctor` ; use a physical Android phone if the emulator is slow.

7.4 Team working agreement

- **Communication:** a WhatsApp group for daily updates and quick questions. We move to Discord for calls or screen sharing when needed.
- **Meetings:** at least one team meeting per week (in person or on Discord) to review progress and plan the next tasks, plus a short check before each assignment deadline.
- **Code workflow:** the `main` branch always contains working code. Each member works on a feature branch and merges through a pull request that another member reviews. Everyone pulls the latest changes before pushing.
- **Merge conflicts:** the members whose changes conflict resolve it together on a call, then run the app before merging. **Missed tasks:** a member who cannot finish a task tells the group on WhatsApp at least 24 hours before it is due. The group reassigns or shares the task so the deadline is still met.

8. Contribution Table

Member	Tasks completed in this assignment	Share
Muhammad Fuzail	Idea research and discussion; project statement and value proposition; GitHub repository creation and group information; screen flow diagram and wireframes	33.3%

Rabia Rafaqat	Idea research and discussion; functional and non-functional requirements with priorities; planned features (Must / Should / Could) in the README; competitor review	33.3%
Javeria Imran	Idea research and discussion; technology stack breakdown, justification and setup guide; style guide; work plan, risks and working agreement	33.3%
	Total	**100%**

9.References and Tools Used

- Flutter (flutter.dev), Dart (dart.dev), Hive (pub.dev/packages/hive)
- Android Studio and Android documentation: developer.android.com; Git and GitHub: github.com
- Material Design and Icons: m3.material.io, fonts.google.com/icons; Poppins: Google Fonts
- Web Content Accessibility Guidelines (contrast): w3.org/TR/WCAG21; competitor apps reviewed: Skillshare and Udemy
- **AI tools used:** Claude (Anthropic) for research, report drafting, diagrams and wireframe layout; Microsoft Copilot for research and help. All content was reviewed by the group.